

HAS A Relationship

● HAS A relationship is the form of association in which one class acts as a container for the objects of another class.

● HAS A Relationship is classified into 2 types:

a) Composition

b) Aggregation

Composition

Composition is a strong form of association in which the contained object cannot exist without the container

eg>

car has - A engine.

mobile has - A camera

college has A department

Aggregation

Aggregation is a weak form of association in which the contained object can exist without the container

eg>

car has - A driver

mobile has A simcard

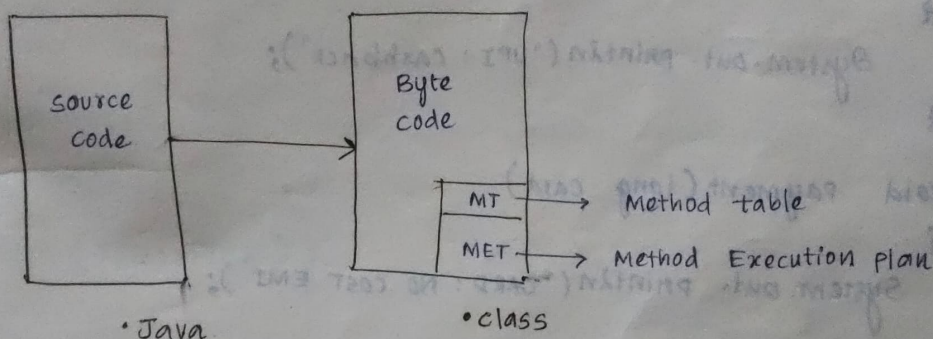
college has a student

The main use of has-A relationship is code

reusability.

Method Binding

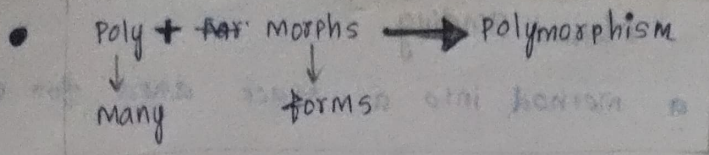
- Before loading a method into a stack area for execution the ^{who} JVM will join the method declaration and method definition this joining activity is known as method binding.
- The method binding activity is always done at the run time by JVM.
- Method binding decisions may be taken either by the compiler at the compile time or by the JVM at the run time.



✓ inheritance + overriding + ^{upcasting} → Method binding decision taken by JVM not by compiler

Polymorphism

- Polymorphism is a concept of one object showing different behaviors in its life time.
- In Java polymorphism is two types
 1. compile time polymorphism / Method overloading
 2. Run Time polymorphism / Method overriding



compile time polymorphism

KEY POINTS

1. Method overloading
2. compiler at compile time
3. Based on Arguments.
4. Static Polymorphism
5. Early binding

① eg) class Flipkart

```
{
    void payments()
    {
        System.out.println("COD");
    }
    void payments(String UPI)
    {
        System.out.println("UPI: cashback");
    }
    void payment(long card)
    {
        System.out.println("CARD: NO COST EMI");
    }
    void payment(String un, String pw)
    {
        System.out.println("Netbanking: 5% Discount");
    }
}
```

```
Flipkart new obj = new Flipkart();
```

```
obj.payments();
```

```
obj.payments("abcd@UPI");
```

```
obj.payment(987654321057L);
```

```
obj.payment("Nandhitha", "12345");
```


2) eg class Irctc

```
void search(int trainNumber)
```

```
{
```

```
    System.out.println("Train Details");
```

```
}
```

```
void search(string src, string dest)
```

```
{
```

```
    System.out.println("Route Details");
```

```
}
```

```
void search(long pr)
```

```
{
```

```
    System.out.println("Ticket Details");
```

```
}
```

```
Irctc ref = new Irctc();
```

```
ref.search(1234);
```

```
ref.search("kss", "MAB");
```

```
ref.search(45123679234);
```

3) eg

```
class TestManager
```

```
{
```

```
void test(int a)
```

```
{
```

```
    System.out.println("Smoke Testing");
```

```
}
```

```
void test(double d)
```

```
{
```

```
    System.out.println("Adhoc Testing");
```

```
}
```

```
void test(string s)
```

```
{
```

```
    System.out.println("Regression Testing");
```

```
}
```

```
TestManager mgr = new TestManager();
```

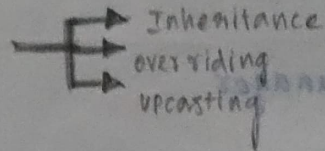
```
mgr.test(25);
```

```
mgr.test(25);
```


Compile Time Polymorphism

- Compile time polymorphism is done with the help of method overloading.
- In case of method overloading the method binding decision are taken by the compiler at the compile time. Hence it is known as compile time polymorphism.
- The compiler takes method binding decisions based on the arguments.
- The binding decision taken by the compiler cannot be changed. Hence it is also known as static polymorphism. (or) static binding.
- In compile time polymorphism the method binding decisions are taken at compile time before program starts execution. Hence it is also known as early binding.

Run time polymorphism



```
① class parent
{
    void marriage()
    {
    }
}

class son extends parent
{
    @Override
    void marriage()
    {
        System.out.println("Love marriage");
    }
}
```



```
class Daughten extends Parent
```

```
{
```

```
    @Override
```

```
    void marriage() marriage()
```

```
{
```

```
    System.out.println("Arrange Marriage");
```

```
}
```

```
}
```

```
public class MainProgram
```

```
{
```

```
    public static void main(String[] args)
```

```
{
```

```
        Parent p;
```

```
        p = new son();
```

```
        p.marriage(); // Love Marriage
```

```
        p = new Daughten();
```

```
        p.marriage(); // Arrange Marriage
```

```
}
```

eg

```
class card
```

```
{
```

```
    void swipe()
```

```
{
```

```
}
```

```
class CreditCard extends card
```

```
{
```

```
    @Override
```

```
    void swipe()
```

```
{
```

```
        s.o.p("due increases");
```

```
}
```

```
class DebitCard extends Card
```

```
{
```

```
    @Override
```

```
    void swipe()
```

```
{
```

```
        s.o.p("Balance reduced");
```

```
}
```



```
class shop
```

```
{
```

```
    static void payment(card c)
```

```
    {
```

```
        if (c != null)
```

```
        {
```

```
            c.swipe();
```

```
        }
```

```
    }
```

```
}
```

```
public class Mainclass
```

```
{
```

```
    P.S.V. m()
```

```
    {
```

```
        DepitcarDepitcard c1 = new Depitcard();
```

```
        Shop.payment(c1);
```

```
        creditcard c2 = new creditcard();
```

```
        Shop.payment(c2);
```

```
    }
```

```
}
```

③ Eg. class Facebook

```
{
```

```
    void display()
```

```
    {
```

```
    }
```

```
}
```

```
class FacebookMobile extends Facebook
```

```
{
```

```
    @Override
```

```
    void display()
```

```
    {
```

```
        S.O.P("Display in mobile");
```

```
    }
```

```
class FacebookWebsite extends Facebook
```

```
{
```

```
    @Override
```

```
    void display()
```

```
    {
```

```
        S.O.P("Display in Browser");
```

```
    }
```

```
}
```



```
class DisplayAdapter
```

```
static void adapt (Facebook Fb)
```

```
{ if (fb != null)
```

```
{ fb.display();
```

```
public class Mainclass
```

```
{ P.S.V.M()
```

```
{ Facebook Mobile fb1 = new FacebookMobile();
```

```
DisplayAdapter . adapt (fb1);
```

```
FacebookWebsite fb2 = new FacebookWebsite();
```

```
DisplayAdapter . adapt (fb2);
```

④ Eg

```
class patient
```

```
{ void takeTreatment()
```

```
Inpatient extends Patient
```

```
class outpatient extends Patient
```

```
{ @Override
```

```
void takeTreatment()
```

```
{ System.out.println("Treatment in Home");
```

```
}
```

```
}
```

```
void takeTreatment()
```

```
{ System.out.println("Treatment in Hospital");
```

```
}
```


class Hospital

```
{
    static void giveTreatment(Patient P)
    {
        if (P != null)
        {
            P.takeTreatment();
        }
    }
}
```

class MainProgram

```
{
    P.S.V.M()
    {
        Inpatient ip = new Inpatient();
        Hospital.giveTreatment(ip); // Treatment in Hospital
        Outpatient op = new Outpatient();
        Hospital.giveTreatment(op); // Treatment in Home
    }
}
```

}

Notes

- Method overriding is used to achieve run time Polymorphism.
- In Run time polymorphism the method binding decisions are taken by the JVM at the run time.
- ~~In~~ JVM takes decision based on the current instance.
- The decisions taken by the JVM can be changed hence it is also known as dynamic polymorphism or dynamic binding.
- The method binding decisions are taken during program execution hence it is also known as late binding.

Compile Time Polymorphism

Done using Method Overloading

Method Binding decisions are taken by compiler at compile time

Compiler takes decision based on arguments.

Method Binding Decisions cannot be changed hence known as static Polymorphism/rebinding is not allowed

Binding Decisions are taken before execution hence known as early binding

Run Time Polymorphism

Done using Method overriding

Method Binding Decisions are taken by JVM at run time

JVM takes decision based on current instance.

Method Binding Decisions can be changed hence known as dynamic Polymorphism/rebinding is allowed

Binding Decisions are taken during execution hence known as late binding.